

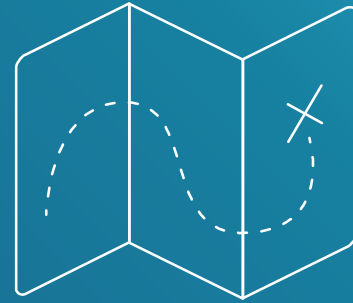
MongoDB + Python Flask Web Framework + REST API + GKE



Belsabel Woldemichael

Contents

- ◇ Introduction
- ◇ Design
- ◇ Implementation
- ◇ Test
- ◇ Enhancement Ideas
- ◇ Conclusion





Introduction

Overview:

- Integrates multiple modern technologies to build a scalable and efficient application.
- Combines MongoDB, Python Flask, and Google Kubernetes Engine (GKE) for a robust solution.

Objective:

- Create a system for managing student records and bookstore inventory.
- Demonstrate the use of microservices architecture and cloud technologies.

Technologies:

- **MongoDB:** NoSQL database for managing dynamic and unstructured data.
- **Python Flask:** Lightweight web framework for developing RESTful APIs.
- **Google Kubernetes Engine (GKE):** Managed Kubernetes service for deploying and scaling containerized applications.

System Components:

- **Student Records Management:**
 - API endpoints for accessing and managing student scores.
- **Bookstore Management:**
 - API endpoints for listing, adding, updating, and deleting books.

Key Features:

- **Pods:** Runs separate applications for student records and bookstore.
- **Services:** Provides external access to the applications.
- **Persistence Volumes:** Ensures data durability with MongoDB.
- **Ingress:** Manages access under a single domain with different paths.
- **ConfigMaps:** Manages configuration dynamically.

Purpose:

- Showcase how modern web technologies and cloud infrastructure can be effectively integrated.
- Provide a practical example of building scalable and maintainable web applications.



Design

- **Architecture:**
 - **Microservices:**
 - **Pod 1:** Student Records Service
 - **Pod 2:** Bookstore Service
- **Components:**
 - **Pods:**
 - **Pod 1:** Handles student records.
 - **Pod 2:** Manages bookstore inventory.
 - **MongoDB:** Central database with persistence volumes for data durability.
 - **Python Flask:** Framework for developing RESTful APIs.
 - **GKE:** Manages deployment, scaling, and operations of containers.

- **Networking:**

- **Services:** Provides access to the pods.
- **Ingress:** Routes requests under a single domain:
 - /studentserver for student records.
 - /bookshelf for bookstore.

- **ConfigMaps:**

- Stores configuration like MongoDB addresses.

- **Data Management:**

- **Persistence Volumes:** Ensures MongoDB data remains intact through restarts.



Implementation and Test

1. Create a GKE Cluster:

```
gcloud container clusters create kubia --num-nodes=1 --machine-type=e2-micro --region=us-west1
```

```
Creating cluster kubia in us-west1... Cluster is being health-checked (master is healthy)...done.  
Created [https://container.googleapis.com/v1/projects/cs570-big-data-429720/zones/us-west1/clusters/kubia].  
To inspect the contents of your cluster, go to: https://console.cloud.google.com/kubernetes/workload_/gcloud/us-west1/kubia?project=cs570-big-data-429720  
kubeconfig entry generated for kubia.  
NAME: kubia  
LOCATION: us-west1  
MASTER_VERSION: 1.29.6-gke.1254000  
MASTER_IP: 34.83.236.190  
MACHINE_TYPE: e2-micro  
NODE_VERSION: 1.29.6-gke.1254000  
NUM_NODES: 3  
STATUS: RUNNING  
belsabelwoldemichael@cs-312169370680-default:~$
```

2. Create Persistent Volume:

```
gcloud compute disks create --size=10GiB --zone=us-west1-a mongodb
```

```
belsabelwoldemichael@cs-312169370680-default:~$ gcloud compute disks create --size=10GiB --zone=us-west1-a mongodb  
WARNING: You have selected a disk size of under [200GB]. This may result in poor I/O performance. For more information, see: https://developers.google.com/compute/docs/disks#performance.  
Created [https://www.googleapis.com/compute/v1/projects/cs570-big-data-429720/zones/us-west1-a/disks/mongodb].  
NAME: mongodb  
ZONE: us-west1-a  
SIZE_GB: 10  
TYPE: pd-standard  
STATUS: READY  
  
New disks are unformatted. You must format and mount a disk before it  
can be used. You can find instructions on how to do this at:  
  
https://cloud.google.com/compute/docs/disks/add-persistent-disk#formatting
```

3. Deploy MongoDB:

Apply the mongodb-deployment.yaml configuration:

```
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mongodb-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mongodb-deployment
spec:
  selector:
    matchLabels:
      app: mongodb
  strategy:
    type: Recreate
  template:
    metadata:
      labels:
        app: mongodb
```

kubectl apply -f mongodb-deployment.yaml

4. Check Deployment Status:

```
belsabelwoldemichael@cs-312169370680-default:~$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
mongodb-deployment-5c589898cb-2jmbf	1/1	Running	0	12m

5. Create MongoDB Service:

Apply the mongodb-service.yaml configuration:

```
kubectl apply -f mongodb-service.yaml
```

```
belsabelwoldemichael@cs-312169370680-default:~$ kubectl apply -f mongodb-service.yaml
service/mongodb-service created
```

```
(ubuntu-project-427623) X +
apiVersion: v1
kind: Service
metadata:
  name: mongodb-service
spec:
  type: LoadBalancer
  ports:
    # service port in cluster
    - port: 27017
      # port to contact inside container
      targetPort: 27017
  selector:
    app: mongodb
```

6. Verify Service Status:

kubectl get svc

```
belsabelwoldemichael@cs-312169370680-default:~$ kubectl get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.3.240.1	<none>	443/TCP	132m
mongodb-service	LoadBalancer	10.3.249.248	34.168.212.202	27017:30117/TCP	39s

7. Test MongoDB Connection:

kubectl exec -it mongodb-deployment-replace-with-your-pod-name -- bash

```
root@mongodb-deployment-5c589898cb-2jmbf:/# mongosh 34.168.212.202
Current Mongosh Log ID: 66adb67323f9c407a5149f47
Connecting to:      mongodb://34.168.212.202:27017/?directConnection=true&appName=mongosh+2.2.10
Using MongoDB:      7.0.12
Using Mongosh:      2.2.10

For mongosh info see: https://docs.mongodb.com/mongodb-shell/

To help improve our products, anonymous usage data is collected and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

-----
The server generated these startup warnings when booting
2024-08-03T04:13:31.648+00:00: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See http://dochub.mongodb.org/core/prodnotes-filesystem
2024-08-03T04:13:35.269+00:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
2024-08-03T04:13:35.270+00:00: vm.max_map_count is too low
-----
```

8. Insert Records into MongoDB:

```
Type ".help" for more information.
> const { MongoClient } = require('mongodb');
undefined
> const url = "mongodb://34.168.212.202:27017/mydb";
undefined
>
> async function run() {
...   try {
seUnifiedTopology: true });t MongoClient.connect(url, { useNewUrlParser: true, u
...     const db = client.db("studentdb");
...
...     const docs = [
...       { student_id: 11111, student_name: "Bruce Lee", grade: 84 },
...       { student_id: 22222, student_name: "Jackie Chan", grade: 93 },
...       { student_id: 33333, student_name: "Jet Li", grade: 88 }
...     ];
...
...     const insertResult = await db.collection("students").insertMany(docs);
...     console.log(`${insertResult.insertedCount} documents were inserted`);
...
...   } catch (err) {
...     console.error(err);
...   }
... }
undefined
>
> run();
Promise {
  <pending>,
  [Symbol(async_id_symbol)]: 50,
  [Symbol(trigger_async_id_symbol)]: 6
}
(node:1191) [MONGODB DRIVER] Warning: useNewUrlParser is a deprecated option: useNewUrlParser has no effect since Node.js Driver version 4.0.0 and will be removed in the next major version
(node:1191) [MONGODB DRIVER] Warning: useUnifiedTopology is a deprecated option: useUnifiedTopology has no effect since Node.js Driver version 4.0.0 and will be removed in the next major version
(node:1191) [MONGODB DRIVER] Warning: useUnifiedTopology is a deprecated option: useUnifiedTopology has no effect since Node.js Driver version 4.0.0 and will be removed in the next major version
3 documents were inserted
{
  _id: new ObjectId('66adc96b5aa842264039a142'),
  student_id: 11111,
  student_name: 'Bruce Lee',
  grade: 84
}
```


Step 2: Modify StudentServer to Fetch Records from MongoDB and Deploy to GKE

```
...   console.log(`${insertResult.insertedCount} documents were inserted`);
...
// Find the student document
const student = await db.collection("students").findOne({ "student_id": student_id });
await client.close();

if (student) {
  // Prepare the response object
  const response = {
    student_id: student.student_id,
    student_name: student.student_name,
    student_score: student.grade
  };

  // Send the response
  res.writeHead(200, { 'Content-Type': 'application/json' });
  res.end(JSON.stringify(response) + '\n');
} else {
  res.writeHead(404);
  res.end("Student Not Found\n");
}
} else {
  res.writeHead(404);
  res.end("Wrong URL, please try again\n");
}
} catch (err) {
  console.error(err);
  res.writeHead(500);
  res.end("Internal Server Error\n");
}
});

// Start the server
server.listen(8080, () => {
  console.log('Server is listening on port 8080');
});
```

1. Create Dockerfile: this will create a newer version of node js, install all the dependencies needed from a json file.

```
GNU nano 6.2 Dockerfile *  
  
FROM node:14  
WORKDIR /app  
COPY package*.json ./  
RUN npm install  
COPY studentServer.js ./  
EXPOSE 8080  
ENTRYPOINT ["node", "studentServer.js"]  
█
```

2. Build Docker Image:

`docker build -t yourdockerhubID/studentserver .`

```
belsabelwoldemichael@cs-312169370680-default:~$ docker build -t yourdockerhubID/studentserver .  
  
tserver .oldemichael@cs-312169370680-default:~$ docker build -t belsabel4/studen  
[+] Building 30.0s (10/10) FINISHED docker:default  
=> [internal] load build definition from Dockerfile 0.1s  
=> [internal] load metadata for docker.io/node:14 0.0s  
=> [internal] load build definition from Dockerfile 0.1s  
=> [internal] load metadata for docker.io/node:14 0.0s
```


3. Push Docker Image to Docker Hub:

`docker push yourdockerhubID/studentserver`

Configure a credential helper to remove this warning: see

<https://docs.docker.com/engine/reference/commandline/login/#credential-stores>

Login Succeeded

belsabelwoldemichael@cs-312169370680-default:~\$ `docker push belsabel4/studentserver`

Using default tag: latest

The push refers to repository [docker.io/belsabel4/studentserver]

18e339622ee7: Pushed

46e8db2eb049: Pushed

9123d67dflee: Pushed

c12208ce35ad: Pushed

0d5f5a015e5d: Mounted from library/node

3c777d951de2: Mounted from library/node

f8a91dd5fc84: Mounted from library/node

cb81227abde5: Mounted from library/node

e01a454893a9: Mounted from library/node

c45660adde37: Mounted from library/node

fe0fb3ab4a0f: Mounted from library/node

f1186e5061f2: Mounted from library/node

b2dba7477754: Mounted from library/node

latest: digest: sha256:b3ae8fe5b19814fc052876abf24b08c6ab7d0a8943937bf7966ff97403808184 size: 3047

belsabelwoldemichael@cs-312169370680-default:~\$

Step 3: Create the Flask Application

1. Create bookshelf.py:

```
@app.route("/book", methods=["POST"])
def add_book():
    book = request.get_json(force=True)
    db.bookshelf.insert_one({
        "book_name": book["book_name"],
        "book_author": book["book_author"],
        "ISBN": book["isbn"]
    })
    return jsonify(message="Book saved successfully!")

@app.route("/book/<id>", methods=["PUT"])
def update_book(id):
    data = request.get_json(force=True)
    response = db.bookshelf.update_one({"_id": ObjectId(id)}, {"$set": {
        "book_name": data["book_name"],
        "book_author": data["book_author"],
        "ISBN": data["isbn"]
    }})
    message = "Book updated successfully!" if response.matched_count else "No book found!"
    return jsonify(message=message)

@app.route("/book/<id>", methods=["DELETE"])
def delete_book(id):
    response = db.bookshelf.delete_one({"_id": ObjectId(id)})
    message = "Book deleted successfully!" if response.deleted_count else "No book found!"
    return jsonify(message=message)

@app.route("/books/delete", methods=["POST"])
def delete_all_books():
    db.bookshelf.delete_many({})
    return jsonify(message="All books deleted!")

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

2. Create requirements.txt

```
GNU nano 6.2 requirements.txt *
Flask==2.2.2
Flask-PyMongo==2.3.0
PyMongo>=3.3
```

3. Dockerize the Flask Application

```
GNU nano 6.2 Dockerfile *
# Set the working directory in the container
FROM python:3.9-alpine

# Set the working directory
WORKDIR /app

# Copy the project files into the container
COPY . /app

# Install dependencies
RUN pip install --upgrade pip
RUN pip install -r requirements.txt

# Set environment variable
ENV PORT=5000

# Expose the port the app runs on
EXPOSE 5000
ENTRYPOINT ["python3"]
CMD ["bookshelf.py"]

^G Help      ^O Write Out ^W Where Is   ^K Cut        ^T Execute   ^C Location
^X Exit      ^R Read File ^N Replace   ^U Paste      ^J Justify    ^_ Go To Line
```

4. Build the Docker Image:

```
belsabelwoldemichael@cs-312169370680-default:~$ nano requirements.txt
elf .belwoldemichael@cs-312169370680-default:~$ docker build -t belsabel4/booksh
[+] Building 99.1s (10/10) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 452B                                0.0s
=> [internal] load metadata for docker.io/library/python:3.9-alpine 0.9s
=> [internal] load .dockerignore                                    0.0s
=> => transferring context: 2B                                       0.0s
=> [stage-1 1/5] FROM docker.io/library/python:3.9-alpine@sha256:2ae59f2 3.1s
=> => resolve docker.io/library/python:3.9-alpine@sha256:2ae59f21f0767b2 0.0s
=> => sha256:2ae59f21f0767b22575b35bd3bddeadcc8cef864d 10.29kB / 10.29kB 0.0s
=> => sha256:b9dbfb82c1778f381862aa819dfd87ec3df04fb4080 1.93kB / 1.93kB 0.0s
```

5. Push the Docker Image to Docker Hub:

`docker push belsabel4/bookshelf`

```
belsabelwoldemichael@cs-312169370680-default:~$ docker push belsabel4/bookshelf
Using default tag: latest
The push refers to repository [docker.io/belsabel4/bookshelf]
28be7e7e13b6: Pushed
7270d5c9fe7a: Pushed
6294ac7d317f: Pushed
594dd4e2a4cf: Pushed
4df756e13a64: Mounted from library/python
6cbe9158af48: Mounted from library/python
9edaad5ce791: Mounted from library/python
3f8d587daf92: Mounted from library/python
78561cef0761: Mounted from library/python
latest: digest: sha256:9f75210a19cecc993c67cfbe966f2932c2da76a098f3d6099279e70cf902fcd9 size: 2210
belsabelwoldemichael@cs-312169370680-default:~$
```

Step 4: Create ConfigMap for both applications to store MongoDB URL and MongoDB name

1. Create a ConfigMap for studentServer

Create a file named studentserver-configmap.yaml with the following content:

```
GNU nano 6.2
apiVersion: v1
kind: ConfigMap
metadata:
  name: studentserver-config
data:
  MONGO_URL: 34.168.212.202
  MONGO_DATABASE: mydb
```


2. Create a ConfigMap for bookshelf

Create a file named bookshelf-configmap.yaml with the following content:

```
GNU nano 6.2
apiVersion: v1
kind: ConfigMap
metadata:
  name: bookshelf-config
data:
  MONGO_URL: 34.168.212.202
  MONGO_DATABASE: mydb
```

Notice:

- The reason for creating these two ConfigMaps is to avoid re-building the Docker image again if the MongoDB pod restarts with a different External-IP.

Step 5: Expose Two Applications Using Ingress with Nginx

1. Create studentserver-deployment.yaml

```
GNU nano 6.2
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
  labels:
    app: studentserver-deploy
spec:
  replicas: 1
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: web
          image: sened/studentserver
          imagePullPolicy: Always
          ports:
            - containerPort: 8080
          env:
            - name: MONGO_URL
              valueFrom:
                configMapKeyRef:
                  name: studentserver-config
                  key: MONGO_URL
            - name: MONGO_DATABASE
              valueFrom:
                configMapKeyRef:
                  name: studentserver-config
                  key: MONGO_DATABASE
```


2. Create bookshelf-deployment.yaml

```
GNU nano 6.2
apiVersion: apps/v1
kind: Deployment
metadata:
  name: bookshelf-deployment
  labels:
    app: bookshelf-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: bookshelf-deployment
  template:
    metadata:
      labels:
        app: bookshelf-deployment
    spec:
      containers:
        - image: belsabel4/bookshelf
          imagePullPolicy: Always
          name: bookshelf-deployment
          ports:
            - containerPort: 5000
          env:
            - name: MONGO_URL
              valueFrom:
                configMapKeyRef:
                  name: bookshelf-config
                  key: MONGO_URL
            - name: MONGO_DATABASE
              valueFrom:
                configMapKeyRef:
                  name: bookshelf-config
                  key: MONGO_DATABASE
```

3. Create studentserver-service.yaml

```
GNU nano 6.2
apiVersion: v1
kind: Service
metadata:
  name: bookshelf-service
spec:
  type: LoadBalancer
  ports:
    - protocol: TCP
      port: 5000
      targetPort: 5000
  selector:
    app: bookshelf-deployment
```

4. Create bookshelf-service.yaml

```
GNU nano 6.2
apiVersion: v1
kind: Service
metadata:
  name: bookshelf-service
spec:
  type: LoadBalancer
  ports:
    - protocol: TCP
      port: 5000
      targetPort: 5000
  selector:
    app: bookshelf-deployment
```

5. Start Minikube

minikube start

```
Welcome to Cloud Shell! Type "help" to get started.
Your Cloud Platform project in this session is set to dulcet-essence-427902-p9.
Use "gcloud config set project [PROJECT_ID]" to change to a different project.
belsabelwoldemichael@cs-312169370680-default:~$ nano studentserver-configmap.yaml
belsabelwoldemichael@cs-312169370680-default:~$ nano bookshelf-configmap.yaml
belsabelwoldemichael@cs-312169370680-default:~$ nano studentserver-deployment.yaml
belsabelwoldemichael@cs-312169370680-default:~$ nano bookshelf-deployment.yaml
belsabelwoldemichael@cs-312169370680-default:~$ nano bookshelf-service.yaml
belsabelwoldemichael@cs-312169370680-default:~$ minikube start
* minikube v1.33.1 on Ubuntu 22.04 (amd64)
  - MINIKUBE_FORCE_SYSTEMD=true
  - MINIKUBE_HOME=/google/minikube
  - MINIKUBE_WANTUPDATENOTIFICATION=false
* Automatically selected the docker driver. Other choices: ssh, none
* Using Docker driver with root privileges
* Starting "minikube" primary control-plane node in "minikube" cluster
* Pulling base image v0.0.44 ...
* Downloading Kubernetes v1.30.0 preload ...
  > preloaded-images-k8s-v18-v1...: 342.90 MiB / 342.90 MiB 100.00% 154.38
  > gcr.io/k8s-minikube/kicbase...: 481.58 MiB / 481.58 MiB 100.00% 98.21 M
* Creating docker container (CPUs=2, Memory=4000MB) ...
* Preparing Kubernetes v1.30.0 on Docker 26.1.1 ...
  - kubelet.cgroups-per-qos=false
  - kubelet.enforce-node-allocatable=""
  - Generating certificates and keys ...
  - Booting up control plane ...
  - Configuring RBAC rules ...
* Configuring bridge CNI (Container Networking Interface) ...
* Verifying Kubernetes components...
  - Using image gcr.io/k8s-minikube/storage-provisioner:v5
* Enabled addons: storage-provisioner, default-storageclass
* Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
```

6. Start Ingress

minikube addons enable ingress

```
belsabelwoldemichael@cs-312169370680-default:~$ minikube addons enable ingress
* ingress is an addon maintained by Kubernetes. For any concerns contact minikube on GitHub.
You can view the list of minikube maintainers at: https://github.com/kubernetes/minikube/blob/master/OWNERS
- Using image registry.k8s.io/ingress-nginx/controller:v1.10.1
- Using image registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.4.1
- Using image registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.4.1
* Verifying ingress addon...
* The 'ingress' addon is enabled
```

7. Create studentserver Related Pods and Start Service

kubectl apply -f studentserver-deployment.yaml

kubectl apply -f studentserver-configmap.yaml

kubectl apply -f studentserver-service.yaml

```
the ingress addon is enabled
belsabelwoldemichael@cs-312169370680-default:~$ kubectl apply -f studentserver-deployment.yaml
kubectl apply -f studentserver-configmap.yaml
kubectl apply -f studentserver-service.yaml
deployment.apps/web created
configmap/studentserver-config created
```

8. Create bookshelf Related Pods and Start Service

```
kubectl apply -f bookshelf-deployment.yaml
kubectl apply -f bookshelf-configmap.yaml
kubectl apply -f bookshelf-service.yaml
```

```
belsabelwoldemichael@cs-312169370680-default:~$ kubectl apply -f bookshelf-deployment.yaml
kubectl apply -f bookshelf-configmap.yaml
kubectl apply -f bookshelf-service.yaml
deployment.apps/bookshelf-deployment created
configmap/bookshelf-config created
service/bookshelf-service created
belsabelwoldemichael@cs-312169370680-default:~$
```

9. Check if All Pods are Running Correctly

```
kubectl get pods
```

```
belsabelwoldemichael@cs-312169370680-default:~$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
bookshelf-deployment-688c67bf95-7qdm7	1/1	Running	0	3m41s
web-797d584f49-pqf8z	1/1	Running	0	4m6s

10. Create Ingress Service YAML File

Create a file named `studentservermongoIngress.yaml` with the following content:

```
GNU nano 6.2 studentservermongoIngress.yaml *
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: server
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /$2
spec:
  rules:
    - host: cs571.project.com
      http:
        paths:
          - path: /studentserver(/|$) (.*)
            pathType: Prefix
            backend:
              service:
                name: web
                port:
                  number: 8080
          - path: /bookshelf(/|$) (.*)
            pathType: Prefix
```


11. Create the Ingress Service

```
kubectl apply -f studentservermongoIngress.yaml
```

```
belsabelwoldemichael@cs-312169370680-default:~$ kubectl apply -f studentservermongoIngress.yaml
ingress.networking.k8s.io/server created
belsabelwoldemichael@cs-312169370680-default:~$
```

12. Check if Ingress is Running

```
kubectl get ingress
```

Wait until you see the Address, then move forward.

```
belsabelwoldemichael@cs-312169370680-default:~$ kubectl get ingress
```

NAME	CLASS	HOSTS	ADDRESS	PORTS	AGE
server	<none>	cs571.project.com	192.168.49.2	80	14m

13. Add Address to /etc/hosts

```
vi /etc/hosts
```

```
#10.0.0.3          myfriend
#
# According to RFC 1918, you can use the following IP networks for private
# nets which will never be connected to the Internet:
#
#      10.0.0.0      -   10.255.255.255
#      172.16.0.0    -   172.31.255.255
#      192.168.0.0   -   192.168.255.255
#
# In case you want to be able to connect directly to the Internet (i.e. not
# behind a NAT, ADSL router, etc...), you need real official assigned
# numbers. Do not try to invent your own network numbers but instead get one
# from your network provider (if any) or from your regional registry (ARIN,
# APNIC, LACNIC, RIPE NCC, or AfrinIC.)
#
169.254.169.254 metadata.google.internal metadata
192.168.49.2 cs571.project.com
█
```



Test

1. Access a student's score

```
belsabelwoldemichael@cs-312169370680-default:~$ curl cs571.project.com/studentserver/api/score?student_id=11111  
{ "student_id": 11111, "student_name": "Bruce Lee", "student_score": 84 }
```

```
belsabelwoldemichael@cs-312169370680-default:~$ curl cs571.project.com/studentserver/api/score?student_id=22222  
{ "student_id": 22222, "student_name": "Jackie Chen", "student_score": 93 }
```

2. Add a book

```
belsabelwoldemichael@cs-312169370680-default:~$ curl -v -X POST -d '{"book_name": "cloud computing", "book_author": "unknown", "isbn": "123456"}'  
http://cs571.project.com/bookshelf/book  
{  
  "message": "Book saved successfully!"  
}
```

3. Update a book

```
belsabelwoldemichael@cs-312169370680-default:~$ curl -X PUT -d '{"book_name":"123Updatedname","book_author":"butcher","isbn":"123Updated"}' http://cs571.project.com/bookshelf/book/66a9b4427947edff15237311
{"message": "Book updated successfully!"}
```

4. List all Books

```
belsabelwoldemichael@cs-312169370680-default:~$ curl cs571.project.com/bookshelf/books
[
  {
    "Book Author": "unknown",
    "Book Name": "cloud computing",
    "ISBN": "123456",
    "id": "66a9b4427947edff15237311"
  }
]
```

5. Delete a book

```
belsabelwoldemichael@cs-312169370680-default:~$ curl -X DELETE cs571.project.com/bookshelf/book/66a9b4427947edff15237311  
{  
  "message": "Book deleted successfully!"  
}
```




Enhancement Ideas

Add User Authentication:

- Implement authentication and authorization for API access to secure sensitive operations.

Improve API Documentation:

- Use tools like Swagger or OpenAPI for interactive and detailed API documentation.

Optimize MongoDB Queries:

- Analyze and optimize queries to enhance database performance and reduce response times.

Add Data Validation:

- Introduce data validation rules to ensure data integrity and prevent invalid data entry.



Conclusion

Integration:

- Successfully integrates MongoDB, Python Flask, and Google Kubernetes Engine (GKE) for a scalable application.

Modularity:

- Divides application into distinct services (student records and bookstore) for better management and scalability.

Robustness:

- Utilizes persistence volumes and ConfigMaps to ensure data integrity and configuration flexibility.

Access and Management:

- Provides a unified domain with distinct paths for accessing different services, simplifying user interaction.

References

Mongo DB

REST API

Python Flask Web Framework

Github link

https://github.com/BelsabelTekle/Cloud_Computing/tree/main/Kubernetes/MongoDB%20+%20Python%20Flask%20Web%20Framework%20+%20REST%20API%20+%20GKE